

AFH

Pre Entrenamiento de IA



Mario Coca Atienza



Índice

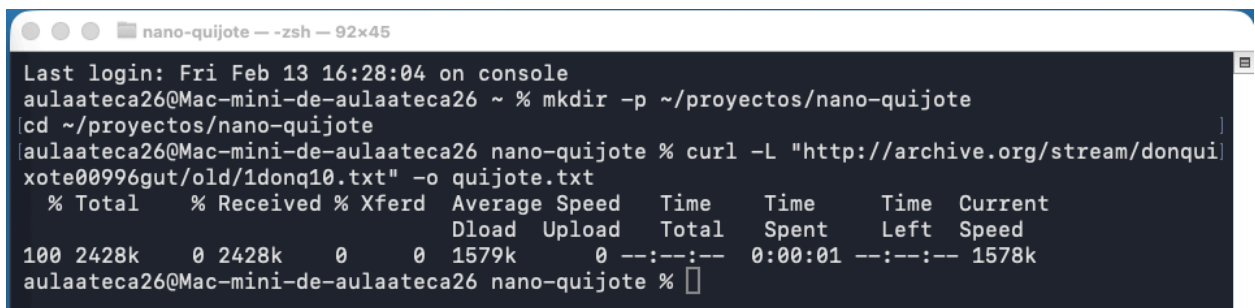
Práctica	2
Paso 1: Preparación del corpus para el pre-entrenamiento	2
Paso 2: Ejecución del script de entrenamiento	2
Paso 3: Revisión del script de entrenamiento	3
Paso 4: Bloque final del script: guardado del modelo y generación de texto	4
Paso 5: Ejecución completa del entrenamiento y prueba de generación	6

Práctica

Paso 1: Preparación del corpus para el pre-entrenamiento

En esta captura se muestra el inicio del proceso de pre-entrenamiento creando el entorno donde se almacenará. Desde la terminal, se genera una carpeta llamada nano-quijote dentro del directorio de proyectos y se accede a ella. Una vez dentro, se descarga mediante curl una copia en texto plano del Quijote desde un repositorio público.

La terminal muestra el progreso de la descarga y confirma que el archivo quijote.txt se ha guardado correctamente. Este archivo será el punto de partida para construir un corpus literario que se utilizará en el pre-entrenamiento del modelo, permitiéndole aprender patrones de lenguaje a partir de un texto extenso y coherente.



```
nano-quijote -- zsh -- 92x45
Last login: Fri Feb 13 16:28:04 on console
aulaateca26@Mac-mini-de-aulaateca26 ~ % mkdir -p ~/proyectos/nano-quijote
cd ~/proyectos/nano-quijote
aulaateca26@Mac-mini-de-aulaateca26 nano-quijote % curl -L "http://archive.org/stream/donqui
xote00996gut/old/1donq10.txt" -o quijote.txt
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100 2428k    0 2428k    0     0  1579k      0 --:--:--  0:00:01 --:--:-- 1578k
aulaateca26@Mac-mini-de-aulaateca26 nano-quijote %
```

Paso 2: Ejecución del script de entrenamiento

En esta captura se muestra el proceso de entrenamiento del modelo utilizando el archivo quijote.txt como corpus. Desde la terminal, se ejecuta el script entrenar.py, que inicia el entrenamiento sobre el dispositivo MPS, optimizado para equipos Apple.

La salida muestra cómo el modelo va reduciendo progresivamente las pérdidas de entrenamiento y validación a lo largo de las iteraciones, lo que indica que está aprendiendo patrones del texto. Al finalizar, el script genera un fragmento de texto como muestra, que aunque aún es incoherente, refleja que el modelo empieza a imitar el estilo de escritura del Quijote.

```

((venv-nano-quijote) aulaateca26@Mac-mini-de-aulaateca26 nano-quijote % nano entrenar.py
(venv-nano-quijote) aulaateca26@Mac-mini-de-aulaateca26 nano-quijote % cd ~/proyectos/nano-q
uijote
python entrenar.py

Usando device: mps
Iniciando entrenamiento en mps...
Iteración 0: loss train 4.6951, loss val 4.6886
Iteración 500: loss train 2.2466, loss val 2.2365
Iteración 1000: loss train 1.8719, loss val 1.8713
Iteración 1500: loss train 1.7045, loss val 1.7282
Iteración 2000: loss train 1.5994, loss val 1.6462
Iteración 2500: loss train 1.5296, loss val 1.5968
Iteración 3000: loss train 1.4791, loss val 1.5531
Iteración 3500: loss train 1.4339, loss val 1.5198
Iteración 4000: loss train 1.3948, loss val 1.5019
Iteración 4500: loss train 1.3694, loss val 1.4797

--- ¡Entrenamiento completado! Generando texto... ---

without
ducheded to him to a lous her expectites smade of all that stow Moriestor's
true, Seristiggles for Mancha (who encesty Princheding to
what, that you will was surm and very about yat to be so quited
pleace in good uponinganced, I conce to very immenshinged to over
thit beaves, it was to Sparant Pettin, I had be scounced hide of the
serves like Don Quiteat contry. Don Fante that the whole head to
all put the certables; like no would, cart as which not are
seeing afterly plaing a didle mu
(venv-nano-quijote) aulaateca26@Mac-mini-de-aulaateca26 nano-quijote % █

```

Paso 3: Revisión del script de entrenamiento

En esta captura se muestra el archivo `entrenar.py` abierto en el editor. Aquí se definen los componentes esenciales del modelo y del proceso de pre-entrenamiento. El script establece los hiper parámetros principales (tamaño de batch, longitud de bloque, número de iteraciones y tasa de aprendizaje) y selecciona automáticamente el dispositivo de cómputo, priorizando MPS para equipos Apple.

También se observa la parte encargada de cargar el texto del Quijote, extraer todos los caracteres únicos y construir el vocabulario. A partir de ahí, el script genera las funciones de codificación y decodificación, y convierte el corpus en tensores que luego se dividen en datos de entrenamiento y validación.



```
UW PICO 5.09 File: entrenar.py

import torch
import torch.nn as nn
from torch.nn import functional as F

# --- Hiperparámetros ---
batch_size = 32
block_size = 128
max_iters = 5000
eval_interval = 500
learning_rate = 3e-4

# <<< CAMBIO IMPORTANTE PARA MAC MINI >>>
if hasattr(torch.backends, "mps") and torch.backends.mps.is_available():
    device = "mps"
elif torch.cuda.is_available():
    device = "cuda"
else:
    device = "cpu"
print("Usando device:", device)

n_embd = 256
n_head = 4
n_layer = 4
dropout = 0.2

# --- Carga y tokenización de datos ---
with open('quijote.txt', 'r', encoding='utf-8') as f:
    text = f.read()

chars = sorted(list(set(text)))
vocab_size = len(chars)
stoi = { ch:i for i,ch in enumerate(chars) }
itos = { i:ch for i,ch in enumerate(chars) }

encode = lambda s: [stoi[c] for c in s]
decode = lambda l: ''.join([itos[i] for i in l])

data = torch.tensor(encode(text), dtype=torch.long)
n = int(0.9*len(data))
train_data = data[:n]
```

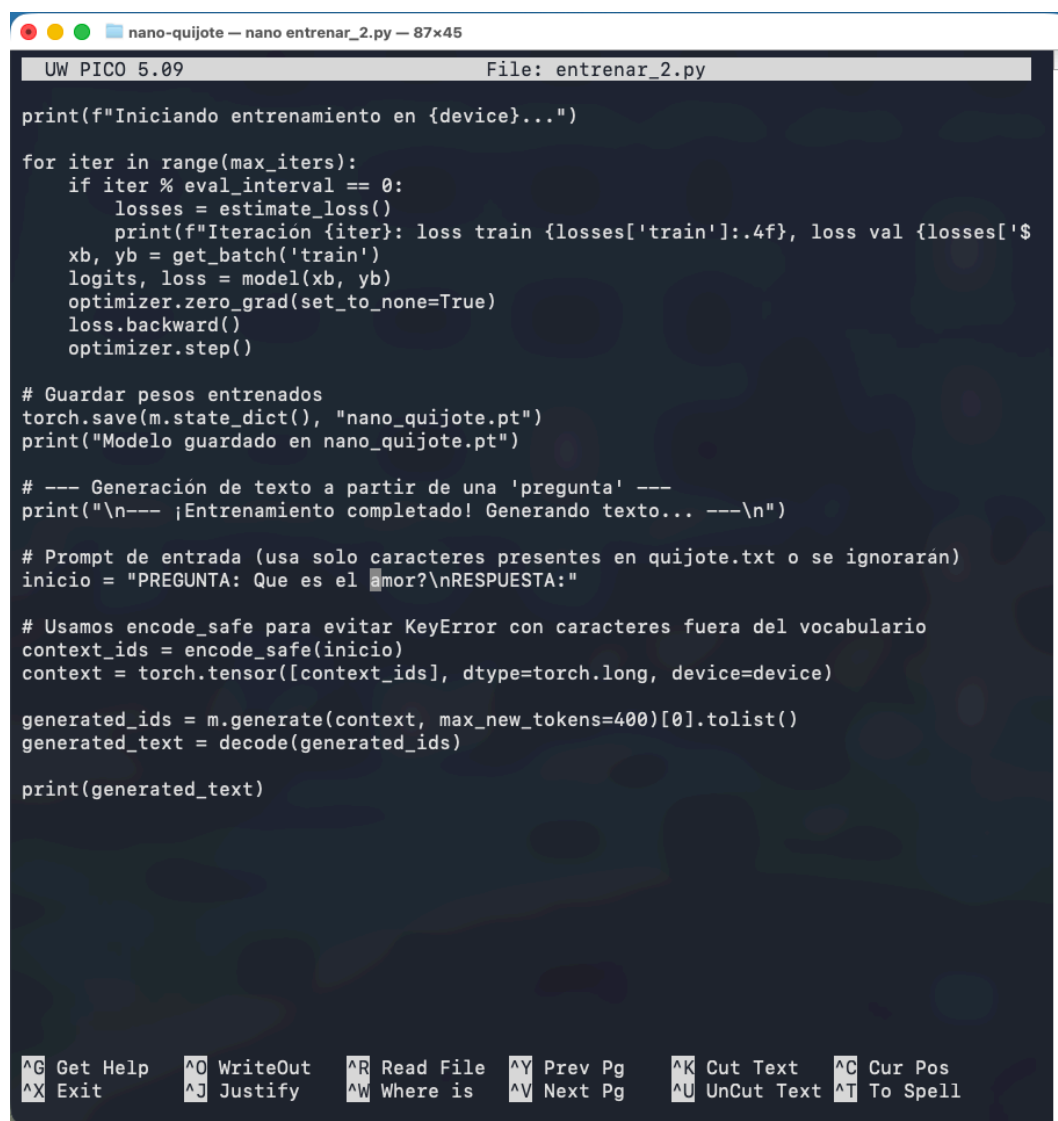
^G Get Help ^O WriteOut ^R Read File ^Y Prev Pg ^K Cut Text ^C Cur Pos
^X Exit ^J Justify ^W Where is ^V Next Pg ^U UnCut Text ^T To Spell

Paso 4: Bloque final del script: guardado del modelo y generación de texto

En esta captura se muestra la parte final del archivo `entrenar_2.py`, donde se completa el ciclo de entrenamiento y se añaden dos pasos clave: guardar los pesos del modelo y generar texto a partir de un prompt.

El código primero imprime el inicio del entrenamiento y ejecuta el bucle principal, donde en cada iteración se calcula la pérdida, se hace retropropagación y se actualizan los parámetros. Una vez alcanzado el número máximo de iteraciones, el modelo se guarda en el archivo nano_quijote.pt, lo que permite reutilizarlo sin volver a entrenarlo desde cero.

Después, el script genera un texto de ejemplo usando un prompt inicial (“PREGUNTA: Qué es el amor? RESPUESTA:”). Para evitar errores con caracteres no presentes en el vocabulario, se utiliza una función de codificación segura. El modelo produce una continuación de texto que se decodifica y se muestra por pantalla, permitiendo evaluar de forma rápida cómo escribe tras el pre-entrenamiento.



```
print(f"Iniciando entrenamiento en {device}...")

for iter in range(max_iters):
    if iter % eval_interval == 0:
        losses = estimate_loss()
        print(f"Iteración {iter}: loss train {losses['train']:.4f}, loss val {losses['$
xb, yb = get_batch('train')
logits, loss = model(xb, yb)
optimizer.zero_grad(set_to_none=True)
loss.backward()
optimizer.step()

# Guardar pesos entrenados
torch.save(m.state_dict(), "nano_quijote.pt")
print("Modelo guardado en nano_quijote.pt")

# --- Generación de texto a partir de una 'pregunta' ---
print("\n--- ¡Entrenamiento completado! Generando texto... ---\n")

# Prompt de entrada (usa solo caracteres presentes en quijote.txt o se ignorarán)
inicio = "PREGUNTA: Que es el amor?\nRESPUESTA:"

# Usamos encode_safe para evitar KeyError con caracteres fuera del vocabulario
context_ids = encode_safe(inicio)
context = torch.tensor([context_ids], dtype=torch.long, device=device)

generated_ids = m.generate(context, max_new_tokens=400)[0].tolist()
generated_text = decode(generated_ids)

print(generated_text)
```

Paso 5: Ejecución completa del entrenamiento y prueba de generación

Esta captura muestra el resultado de ejecutar el script `entrenar_2.py` dentro del entorno virtual `venv-nano-quijote`. El modelo se entrena sobre el dispositivo MPS, y la terminal refleja cómo las pérdidas de entrenamiento y validación disminuyen progresivamente a lo largo de las iteraciones, lo que indica que el modelo está aprendiendo.

Una vez finalizado el entrenamiento, se guarda el modelo en el archivo `nano_quijote.pt` y se genera un texto de ejemplo a partir de un prompt: “¿Qué es el amor?”. La respuesta generada, aunque incoherente, muestra que el modelo ha comenzado a producir texto con estructura y estilo similares al corpus original.

La captura también incluye un intento fallido de ejecutar el comando `drive`, lo que sugiere que se intentaba acceder a una herramienta externa no instalada. En conjunto, esta imagen confirma que el ciclo de entrenamiento se ha completado y que el modelo está operativo para generar texto.

```
(venv-nano-quijote) aulaateca26@Mac-mini-de-aulaateca26 nano-quijote % cd ~/proyectos/nano-quijote
source venv-nano-quijote/bin/activate
python entrenar_2.py

Usando device: mps
Iniciando entrenamiento en mps...
Iteración 0: loss train 4.7645, loss val 4.7756
Iteración 500: loss train 2.2365, loss val 2.2108
e
Iteración 1000: loss train 1.8528, loss val 1.8652
Iteración 1500: loss train 1.6936, loss val 1.7298
Iteración 2000: loss train 1.5962, loss val 1.6468
Iteración 2500: loss train 1.5297, loss val 1.6062
Iteración 3000: loss train 1.4758, loss val 1.5616
Iteración 3500: loss train 1.4425, loss val 1.5331
Iteración 4000: loss train 1.4136, loss val 1.5052
Iteración 4500: loss train 1.3877, loss val 1.4793
Modelo guardado en nano_quijote.pt

--- ¡Entrenamiento completado! Generando texto... ---

PREGUNTA: Que es el amor?
RESPUESTA:

"Will there Riconfles, it what Sancho, "go tie,- it hour
light returned subdings in the king, that the judge musking sizal."

Fining in some datifside, "Need to the
"by who says!" said Sancho Mairallon's.

"Yexak it is proppared they deperds are and plet assom what
an
among; in the miserud is the gier of them Comilian, when he had . At the
othermour and without or in this own endugnifice him in
(venv-nano-quijote) aulaateca26@Mac-mini-de-aulaateca26 nano-quijote % drive
zsh: command not found: drive
(venv-nano-quijote) aulaateca26@Mac-mini-de-aulaateca26 nano-quijote %
```